Universidade do Algarve

Trabalho 1:

Loja Virtual

Tecnologias de Mercado 2025-2026

Alexandre Pita e Daniel Clímaco

Enunciado

Introdução

O objetivo deste trabalho é desenvolver uma loja virtual em Java, com a qual o utilizador interage através da consola. O sistema tem 2 tipos de utilizador: **Cliente** e **Dono da Loja**.

- O **Cliente** pode consultar, comprar e devolver produtos da loja.
- O **Dono da Loja** consegue mudar o nome da loja e gerir os produtos.

O programa deve ter apenas 1 loja e pode ter vários clientes. A loja apenas tem 1 dono e pode ter vários produtos.

O aluno é livre de escolher o tema da loja e os respetivos produtos, desde que a implementação cumpra todos os requisitos definidos no presente enunciado.

Loja

A **loja**, representada pela classe ***Store***, tem um nome, exatamente 1 dono e um **inventário**, que representa o *stock* da loja.

A loja deve ter 2 vistas: **vista do cliente** e **vista do dono**. Ambas as vistas devem ter o nome da loja, o utilizador atual (com nome, se é cliente ou dono, e saldo), a lista de produtos e a lista de ações possíveis para o utilizador atual.

A lista de produtos deve conter um número (índice), começando em 1. Deve também conter o stock atual e o preço por unidade do produto.

A lista de ações deve conter para cada ação uma descrição do que ela faz. Para executar uma ação o programa deve ler do utilizador o nome da ação e em seguida, se assim se aplicar, pedir os dados necessários. Certifique-se que são apresentados os devidos erros caso o utilizador forneça dados inválidos, por exemplo índice de produto inválido. Em caso de dados inválidos, a ação deve ser cancelada e a devida mensagem de erro deve ser apresentada. Caso a ação termine com sucesso, deve ser apresentada a devida mensagem de sucesso.

As ações que um cliente pode executar são:

- *store*
- *buy*
- *inv*
- *return*
- *client*
- *owner*
- *register*

As ações que o dono da loja pode executar são:

- *store*
- *add*
- *remove*
- *edit*
- *stock*
- *rename*
- *client*
- *regi*ster

O programa deve impedir o utilizador de executar uma ação que o tipo de utilizador não possa executar.

```
=== Fruit Shop ===
Current user: John (Client) (100.0€)
Products:
1 - Apple x20 (3.0€)
2 - Banana x20 (5.0€)
3 - Strawberry x20 (2.0€)
------
Actions:
  store - Show store view
  buy - Buy a product
  inv - List client inventory items
  return - Return a product that was bought
  client - Switch to another client
  owner - Switch to owner of store
  register - Add new client
Enter action: add
Invalid action "add".

Enter action:
```

Vista do Cliente

```
=== Fruit Shop ===
Current user: John (Client) (100.0€)
Products:
1 - Apple x10 (3.0€)
2 - Banana x10 (5.0€)
3 - Strawberry x10 (2.0€)
------
Actions:
  store - Show store view
  buy - Buy a product
  inv - List client inventory items
  return - Return a product that was bought
  client - Switch to another client
  owner - Switch to owner of store
  register - Add new client
Enter action:
```

Vista do Dono da Loja

```
=== Fruit Shop ===
Current user: John (Owner) (100.0€)
Products:
1 - Apple x10 (3.0€)
2 - Banana x10 (5.0€)
3 - Strawberry x10 (2.0€)
------
Actions:
  store - Show store view.
  add - Add a product
  remove - Remove a product
  edit - Edit a product
  stock - Increase stock of a product
  rename - Change store name
  client - Switch to a client
  register - Add new client
Enter action:
```

Ações

Ver loja (*store*)

Mostra a vista da loja para o utilizador atual. Esta ação não requer dados.

```
Enter action: store

=== Fruit Shop ===
Current user: John (Client) (100.0€)
Products:
1 - Apple x20 (3.0€)
2 - Banana x20 (5.0€)
3 - Strawberry x20 (2.0€)
------
Actions:
  store - Show store view
  buy - Buy a product
  inv - List client inventory items
  return - Return a product that was bought
  client - Switch to another client
  owner - Switch to owner of store
  register - Add new client
Enter action:
```

Comprar (*buy*)

Compra um produto da loja. Após executar a ação, o programa deve pedir ao utilizador o índice (número na lista) do produto e em seguida a quantidade que deseja comprar. Certifique-se de que o programa apresenta os devidos erros caso o cliente não tenha saldo suficiente ou a loja não tenha stock suficiente. Após a compra com sucesso, deve ser mostrada a vista de loja.

```
=== Fruit Shop ===
Current user: John (Client) (100.0€)
Products:
1 - Apple x20 (3.0€)
2 - Banana x20 (5.0€)
3 - Strawberry x20 (2.0€)
------
Actions:
  store - Show store view
  buy - Buy a product
  inv - List client inventory items
  return - Return a product that was bought
  client - Switch to another client
  owner - Switch to owner of store
  register - Add new client
Enter action: buy
Enter product index: 1
Enter quantity: 10
Successfully bought 10 of "Apple".

=== Fruit Shop ===
Current user: John (Client) (70.0€)
Products:
1 - Apple x10 (3.0€)
2 - Banana x20 (5.0€)
3 - Strawberry x20 (2.0€)
------
Actions:
  store - Show store view
  buy - Buy a product
  inv - List client inventory items
  return - Return a product that was bought
  client - Switch to another client
```

```
  owner - Switch to owner of store
  register - Add new client
Enter action: buy
Enter product index: 2
Enter quantity: 20
You don't have enough balance to buy 20 of "Banana".
You need at least more 30.0€.

Enter action: buy
Enter product index: 3
Enter quantity: 25
Not enough stock to buy 25 of Strawberry.

Enter action: buy
Enter product index: 4
Enter quantity: 2
Invalid product index.

Enter action:
```

Inventário (*inv*)

Mostra o inventário do cliente. Deve mostrar a lista dos produtos comprados, com o índice, nome, quantidade e preço de cada um. O preço deve ser o preço pelo qual o produto foi comprado. Caso o preço do produto mude na loja, este não deve mudar no inventário do cliente.

```
Enter action: inv
John's inventory:
1 - Apple x10 (3.0€)
2 - Strawberry x8 (2.0€)

Enter action:
```

Devolver (*return*)

Devolve um produto que o cliente tenha comprado. Deve mostrar a lista de produtos no inventário do cliente. Em seguida, deve pedir o índice do produto a devolver e depois deve devolver o produto à loja e o dinheiro deve ser devolvido ao cliente. Certifique-se que apresenta os devidos erros caso o produto tenha sido removido da loja ou caso o dono da loja não tenha saldo suficiente para efetuar o reembolso. Após a devolução, deve mostrar a vista de cliente da loja.

```
Enter action: return
John's inventory:
1 - Apple x4 (3.0€)
2 - Strawberry x8 (2.0€)

Enter product index: 1
Successfully returned 4 of product "Apple".


=== Fruit Shop ===
Current user: John (Client) (84.0€)
Products:
1 - Apple x20 (3.0€)
2 - Banana x20 (5.0€)
3 - Strawberry x12 (2.0€)
------
Actions:
  store - Show store view
  buy - Buy a product
  inv - List client inventory items
  return - Return a product that was bought
  client - Switch to another client
  owner - Switch to owner of store
  register - Add new client
Enter action:
```

Trocar de cliente (*client*)

Troca para outro cliente. Deve mostrar a lista de clientes, incluindo o nome e saldo de cada um. Após trocar, deve mostrar a vista de cliente da loja.

```
Enter action: client
Clients:
1 - John (100.0€)
2 - Alice (100.0€)

Enter client index: 2
Switched to client "Alice".


=== Fruit Shop ===
Current user: Alice (Client) (100.0€)
Products:
1 - Apple x20 (3.0€)
2 - Banana x20 (5.0€)
3 - Strawberry x20 (2.0€)
------
Actions:
  store - Show store view
  buy - Buy a product
  inv - List client inventory items
  return - Return a product that was bought
  client - Switch to another client
  owner - Switch to owner of store
  register - Add new client
Enter action:
```

Trocar para o dono da loja (*owner*)

Troca para o dono da loja. Após trocar, deve mostrar a vista de dono da loja.

```
Enter action: owner
Switched to store owner.

=== Fruit Shop ===
Current user: John (Owner) (100.0€)
Products:
1 - Apple x20 (3.0€)
2 - Banana x20 (5.0€)
3 - Strawberry x20 (2.0€)
------
Actions:
  store - Show store view
  add - Add a product
  remove - Remove a product
  edit - Edit a product
  stock - Increase stock of a product
  rename - Change store name
  client - Switch to a client
  register - Add new client
Enter action:
```

Adicionar cliente (*register*)

Adiciona um novo cliente. Deve pedir o nome do novo cliente. O saldo deve ser sempre inicializado com 100€.

```
Enter action: register
Enter new client name: Bob
Client "Bob" added successfully.

Enter action:
```

Adicionar produto (*add*)

Adiciona um produto à loja. Deve pedir o nome e preço do produto a adicionar. O stock do produto deve ser sempre inicializado com 20. Após adicionar o produto, deve mostrar a vista de dono da loja.

```
Enter action: add
Enter new product name: Melon
Enter new product price: 7,5
Product "Melon" added successfully.

=== Fruit Shop ===
Current user: John (Owner) (100.0€)
Products:
1 - Apple x20 (3.0€)
2 - Banana x20 (5.0€)
3 - Strawberry x20 (2.0€)
4 - Melon x20 (7.5€)
------
Actions:
  store - Show store view
  add - Add a product
  remove - Remove a product
  edit - Edit a product
  stock - Increase stock of a product
  rename - Change store name
  client - Switch to a client
  register - Add new client
Enter action:
```

Remover produto (*remove*)

Remove um produto da loja. Deve pedir o índice do produto a remover. Após remover o produto, deve mostrar a vista de dono da loja.

```
Enter action: remove
Enter product index: 2
Product removed successfully.

=== Fruit Shop ===
Current user: John (Owner) (100.0€)
Products:
1 - Banana x20 (5.0€)
2 - Strawberry x20 (2.0€)
------
Actions:
  store - Show store view
  add - Add a product
  remove - Remove a product
  edit - Edit a product
  stock - Increase stock of a product
  rename - Change store name
  client - Switch to a client
  register - Add new client
Enter action:
```

Adicionar stock (*stock*)

Incrementa o stock de um produto da loja. Deve pedir o índice do produto e a quantidade de stock a adicionar. Após adicionar stock, deve mostrar a vista de dono da loja.

```
Enter action: stock
Enter product index: 1
Enter how much stock to add: 10
Increased stock of product by 10.

=== Fruit Shop ===
Current user: John (Owner) (100.0€)
Products:
1 - Apple x30 (3.0€)
2 - Banana x20 (5.0€)
3 - Strawberry x20 (2.0€)
------
Actions:
  store - Show store view
  add - Add a product
  remove - Remove a product
  edit - Edit a product
  stock - Increase stock of a product
  rename - Change store name
  client - Switch to a client
  register - Add new client
Enter action:
```

Renomear loja (*rename*)

Altera o nome da loja. Deve pedir o novo nome da loja e em seguida mostrar a vista de dono da loja.

```
Enter action: rename
Enter new store name: Amazing Fruit Shop
Store renamed to "Amazing Fruit Shop".


=== Amazing Fruit Shop ===
Current user: John (Owner) (100.0€)
Products:
1 - Apple x20 (3.0€)
2 - Banana x20 (5.0€)
3 - Strawberry x20 (2.0€)
------
Actions:
  store - Show store view
  add - Add a product
  remove - Remove a product
  edit - Edit a product
  stock - Increase stock of a product
  rename - Change store name
  client - Switch to a client
  register - Add new client
Enter action:
```

Inventários

Um inventário, representado pela classe *Inventory*, permite armazenar e gerir produtos. Um inventário deve ser capaz de armazenar até um máximo de 100 produtos.

O inventário deve conter métodos para adicionar, remover e obter produtos.

Utilize esta classe para armazenar os produtos da loja e os produtos que cada cliente comprou.

Interface da Loja

Crie uma classe *StoreInterface* que pede e lê dados do utilizador, usando uma instância da loja para executar as ações pedidas pelo utilizador.

Requisitos

- O trabalho deve ser implementado em Java, com interação do utilizador através da consola.
- Deve utilizar apenas os conhecimentos e conteúdos lecionados até ao momento nas aulas.
- O código deve seguir boas práticas de programação orientada a objetos, utilizando conceitos como classes, herança, encapsulamento e polimorfismo, tal como dado nas aulas.
- A implementação deve ser modular, organizada e de fácil manutenção, permitindo expansão futura do sistema.
- A única classe que deve ler input é a classe *StoreInterface*.
- As únicas classes que devem imprimir na consola são as classes *Store* e *StoreInterface*.
- Quando o programa é executado, deve criar uma loja e um dono para a mesma. Deve também criar alguns clientes e adicionar alguns produtos iniciais. O programa deve começar com o primeiro cliente selecionado.
- A classe *Main* deve conter apenas a inicialização da loja, dono, clientes, produtos e interface da loja.

Notas

- Adicione todas as classes, atributos e métodos que achar necessário.
- Adicione comentários onde achar necessário, para ajuda própria e para quem for ler o código.

Entrega e validação

O trabalho deve ser entregue até ao dia **20 de março às 23:59**, na tutoria. Caso o trabalho seja entregue após o prazo, mas com atraso de até 24 horas, a nota do trabalho será penalizada por 2 valores. Entregas com atraso superior a 24 horas não são aceites.

Após a entrega, o trabalho será validado no dia **25 de março** durante a aula das **9:00 às 13:00**.

A validação consiste numa discussão individual sobre o trabalho, para validar a autoria do mesmo, e avaliar a compreensão técnica do que foi desenvolvido. Durante a discussão, o aluno deverá saber explicar o funcionamento do código e as decisões tomadas. Esta etapa poderá impactar a nota final do trabalho.

A avaliação do trabalho terá em conta, além de outros, os seguintes principais critérios:

- Implementação correta das funcionalidades pedidas.
- Cumprimento dos requisitos citados.
- A utilização adequada de boas práticas e conceitos de programação orientada a objetos.
- A organização e modularidade do código.

A implementação de funcionalidades não pedidas não é compensada nem afeta a nota do trabalho.

Bom trabalho!